

E080

AI Engineering with C#

Armored Drill

8 concepts · 12 items · 4 transfer items

What this is. Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

Source. Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	_____	

Items

1.

A developer on a team that moved from Python last quarter writes `var count = 10;` and on the next line `count = "ten";` in Visual Studio with the dark theme. What happens?

- A. A var variable can never be reassigned at all.
- B. A compile-time error, because count was inferred as int.
- C. It compiles but throws an exception at run time.
- D. It runs, and count now holds the text "ten."

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

2.

In a game studio's physics code, reviewed by a senior engineer with ten years of C# experience, Point is a struct. The code runs: `var a = new Point { X = 1 }; var b = a; b.X = 5; Console.WriteLine(a.X);` What prints?

- A. 6
- B. An exception, because structs cannot be copied.
- C. 5
- D. 1

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

3. [TRANSFER — reversed stem / different variable]

Point is now a class. The code runs: `var a = new Point { X = 1 }; var b = a; b.X = 5; Console.WriteLine(a.X);` What prints?

- A. 5
- B. 1
- C. A compile error, because b was never created with new.
- D. 6

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

4.

A help-desk dashboard for a company with 7 support offices computes the average rating: `int total = 7; int count = 2; double avg = total / count;` The ratings come from a customer survey sent every Monday. What is `avg`?

- A. About 0.29
- B. 4.0
- C. 3.0
- D. 3.5

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

5. [TRANSFER — reversed stem / different variable]

With `int total = 7` and `int count = 2`, which expression gives 3.5?

- A. `Math.Round(total / count)`
- B. `(double)total / count`
- C. `total / count * 1.0`
- D. `(double)(total / count)`

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

6.

A Python class `ChatAgent`, written by a contractor who left in March, inherits from both `Logger` and `Retrier`. The team is porting it to C# for a client in the finance sector. How should the C# version be structured?

- A. Implement one interface and inherit from two classes.
- B. C# classes cannot use inheritance, so copy all the code in.
- C. Inherit from at most one base class and implement the other behaviors as interfaces.
- D. Inherit from both `Logger` and `Retrier`, as in Python.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

7. [TRANSFER — reversed stem / different variable]

What does class `Bot : BaseBot, ILogger, IDisposable` declare?

- A. `Bot` inherits from three classes.
- B. `Bot` must be abstract because it lists interfaces.
- C. `BaseBot` inherits from `Bot`.
- D. `Bot` inherits from `BaseBot` and implements `ILogger` and `IDisposable`.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

8.

A recommendation service, deployed on three servers in two regions, stores a confidence threshold that other classes must read. It may only change to a value between 0 and 1. The class file is about 400 lines long. What is the best design?

- A. A private field with no way to read it.
- B. A public property with a private setter, plus a method that validates new values.
- C. A global static variable shared by all classes.
- D. A public field so any class can read and set it.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

9.

A console tool used by a nonprofit's volunteers asks for an age. A user types abc. The volunteers meet on Thursdays. Which approach handles this without crashing?

- A. Use `int.TryParse` and ask again when it returns false.
- B. Cast the string with `(int)input`.
- C. Use `int.Parse` with no try/catch.
- D. Use `Convert.ToInt32`, which returns 0 for bad text.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

10. [TRANSFER — reversed stem / different variable]

After `bool ok = int.TryParse("42x", out int n)`; what are `ok` and `n`?

- A. `ok` is false and `n` is 42.
- B. A `FormatException` is thrown.
- C. `ok` is true and `n` is 42.
- D. `ok` is false and `n` is 0.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

11.

A desktop app for a real-estate office calls a cloud sentiment API when a button is clicked. The handler uses `var result = client.PostAsync(url, body).Result`; and the window freezes for several seconds. The office has 22 agents. What is the best fix?

- A. Make the handler async and await `PostAsync`.
- B. Add `.Wait()` before `.Result` to make sure the call finishes.
- C. Add `Thread.Sleep(2000)` before the call.
- D. Mark the method async; that always makes the request faster.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

12.

A student team at a hackathon sponsored by a local bank hard-coded an AI service API key in Program.cs and pushed it to a public repository. The app won second place. What should they do first?

- A. Make the repository private; that fully solves the problem.
- B. Encode the key in Base64 in the source file.
- C. Revoke and rotate the key, then load the new one from configuration or a secret store.
- D. Delete the key from the latest commit.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	_____	

STOP

Answers and dissection follow. Finish every item and every strip first.

Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

1. Answer B Concept 1

A developer ~~on a team that moved from Python last quarter~~ writes `var count = 10;` and on the next line `count = "ten";` ~~in Visual Studio with the dark theme~~. What happens?

Target word: result of reassigning a var to another type **Decides:** var infers int at compile time; the type cannot change.

A	Absolute language	var variables can be reassigned to values of the same type.
B	Correct	var is statically typed; a string cannot be assigned to an int.
C	Adjacent category	The type check happens at compile time, before anything runs.
D	Confusion set	That is Python's dynamic typing, not C#.

2. Answer D Concept 2

In a game studio's physics code, ~~reviewed by a senior engineer with ten years of C# experience~~, Point is a struct. The code runs: `var a = new Point { X = 1 }; var b = a; b.X = 5; Console.WriteLine(a.X);` What prints?

Target word: value-type copy behavior **Decides:** Point is a struct, so b is an independent copy.

A	Wrong stem number	Adds the two values; assignment replaces, it does not add.
B	Absolute language	Structs are copied on every assignment.
C	Adjacent category	That is class (reference) behavior.
D	Correct	Changing the copy b does not change a.

3. Answer A Concept 2 · transfer

Point is now a class. The code runs: `var a = new Point { X = 1 }; var b = a; b.X = 5; Console.WriteLine(a.X);` What prints?

Target word: reference-type behavior **Decides:** a and b refer to the same object.

A	Correct	Both variables point to one object, so the change shows through a.
B	Inverted operation	That is struct (copy) behavior.
C	Dropped element	<code>b = a</code> is valid; it copies the reference.
D	Wrong stem number	Assignment replaces the value; nothing is added.

4. Answer C Concept 3

A help-desk dashboard ~~for a company with 7 support offices~~ computes the average rating: `int total = 7; int count = 2; double avg = total / count;` ~~The ratings come from a customer survey sent every Monday~~. What is avg?

Target word: value after integer division **Decides:** `total / count` is `int / int`, which truncates to 3 before conversion.

A	Inverted operation	Divides count by total.
B	Adjacent category	Integer division truncates; it does not round.
C	Correct	<code>7 / 2 = 3</code> in integer math; then 3 is stored as 3.0.
D	Dropped element	Ignores that both operands are int.

5. Answer B Concept 3 · transfer

With `int total = 7` and `int count = 2`, which expression gives 3.5?

Target word: choose the conversion that keeps the fraction **Decides:** An operand must be double before the division.

A	Confusion set	Rounds an already-truncated 3.
B	Correct	Casting first makes it double / int, giving 3.5.
C	Adjacent category	The int division happens first: $3 \times 1.0 = 3.0$.
D	Inverted operation	Divides first (3), then casts: 3.0.

6. Answer C Concept 4

A Python class `ChatAgent`, ~~written by a contractor who left in March~~, inherits from both `Logger` and `Retrier`. The team is porting it to C# ~~for a client in the finance sector~~. How should the C# version be structured?

Target word: port multiple inheritance **Decides:** C# allows one base class and many interfaces.

A	Inverted operation	Reverses the rule: many interfaces, one class.
B	Absolute language	Single inheritance is allowed.
C	Correct	C# classes support single inheritance plus multiple interfaces.
D	Confusion set	C# does not allow multiple class inheritance.

7. Answer D Concept 4 · transfer

What does class `Bot : BaseBot, ILogger, IDisposable` declare?

Target word: read a declaration **Decides:** The first entry may be a base class; the I-names are interfaces.

A	Confusion set	C# classes cannot inherit from multiple classes.
B	Adjacent category	Any class can implement interfaces; abstract is a separate choice.
C	Inverted operation	The colon points from the derived class to its base.
D	Correct	One base class followed by interfaces.

8. Answer B Concept 5

A recommendation service, ~~deployed on three servers in two regions~~, stores a confidence threshold that other classes must read. It may only change to a value between 0 and 1. ~~The class file is about 400 lines long~~. What is the best design?

Target word: encapsulation choice **Decides:** Readable by others; changes must be validated.

A	Inverted operation	Blocks the reads the stem requires.
B	Correct	Others can read it; only validated changes get through.
C	Adjacent category	Shares state widely with no control over changes.
D	Dropped element	Drops the validation requirement.

9. Answer A Concept 6

A console tool ~~used by a nonprofit's volunteers~~ asks for an age. A user types abc. ~~The volunteers meet on Thursdays.~~ Which approach handles this without crashing?

Target word: safe numeric parsing **Decides:** Input may not be numeric.

A	Correct	TryParse reports failure without throwing.
B	Adjacent category	A string cannot be cast to int; this will not compile.
C	Dropped element	Parse throws a FormatException on abc.
D	Confusion set	Convert.ToInt32 also throws on non-numeric text; it returns 0 only for null.

10. Answer D Concept 6 · transfer

After `bool ok = int.TryParse("42x", out int n);` what are ok and n?

Target word: TryParse outputs on bad input **Decides:** "42x" is not a valid integer.

A	Dropped element	Ignores that n is reset to 0 on failure.
B	Confusion set	That is int.Parse behavior.
C	Inverted operation	TryParse does not parse a partial number.
D	Correct	TryParse fails and sets the out value to 0.

11. Answer A Concept 7

A desktop app ~~for a real estate office~~ calls a cloud sentiment API when a button is clicked. The handler uses `var result = client.PostAsync(url, body).Result;` and the window freezes for several seconds. ~~The office has 22 agents.~~ What is the best fix?

Target word: fix a blocking call **Decides:** .Result blocks the UI thread during an I/O call.

A	Correct	Awaiting frees the UI thread while the request is in flight.
B	Adjacent category	Still blocks the UI thread.
C	Inverted operation	Adds more blocking instead of removing it.
D	Absolute language	async keeps the app responsive; it does not speed up the network.

12. Answer C Concept 8

A student team at a hackathon ~~sponsored by a local bank~~ hard-coded an AI service API key in Program.cs and pushed it to a public repository. ~~The app won second place.~~ What should they do first?

Target word: remedy for an exposed key **Decides:** A secret was published in source control.

A	Absolute language	The key may already have been copied; it must be rotated.
B	Adjacent category	Encoding is not protection; anyone can decode it.
C	Correct	The old key is compromised; the new one must stay out of source.
D	Dropped element	The key remains in history and is still valid.

Triage grid

Write each item number in the box it belongs to.

	Right	Wrong
Sure	Owned.	Concept inversion — top priority. A wrong mental model; it survives rewording.
Unsure / guess	Recognition, not mastery. Re-drill with the transfer.	Not learned yet.

Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

Trap	What it does	In this drill	Your misses
Dropped element	Leaves out a required term or step.	6	
Wrong stem number	A figure copied from the prompt that is not the answer.	2	
Adjacent category	The right idea in the wrong bucket.	9	
Inverted operation	Direction flipped: add/subtract, cause/effect, higher/lower.	8	
Absolute language	Always, never, guarantee — where judgment applies.	5	
Confusion set	Every option is real; the test is exact role-matching.	6	
Camouflage	Answers a stem fact that was never the question.	0	

Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

#	Concept	Rule	Trap	Tell
1	Static typing and var	C# checks types at compile time; var infers a fixed type from its initializer, it does not make a variable dynamic.	Treating var like a Python variable that can hold any type later.	A variable declared with var and later assigned a different type.
2	Value types vs. reference types	Assigning a struct copies its data; assigning a class instance copies a reference to the same object.	Expecting a struct copy to change the original, or a class copy to be independent.	b = a followed by a change to b and a question about a.
3	Integer division and numeric conversion	int / int truncates before any assignment to double; cast an operand first to keep the fraction.	Assuming assigning to a double variable makes the division fractional.	Two int variables divided and stored in a double.
4	Inheritance and interfaces	A C# class inherits from at most one base class but may implement many interfaces.	Porting Python multiple inheritance directly into C# classes.	A Python class with two or more base classes being translated.
5	Encapsulation with properties	Expose state through properties and methods that enforce rules; keep setters private when changes need validation.	Making fields public for convenience, losing control over valid values.	A value outside code must read but should only change under rules.
6	Safe input parsing	int.TryParse returns false (and sets the out value to 0) on bad input instead of throwing; Parse and Convert throw.	Believing Convert.ToInt32 quietly handles non-numeric text.	Console or form input that may not be a number.
7	async/await for external calls	Await I/O-bound calls such as web or AI service requests; blocking with .Result or .Wait() freezes the calling thread and can deadlock a UI.	Adding .Result or Thread.Sleep to “wait properly.”	A UI that freezes while calling a remote service.
8	Integrating AI services safely	Keep API keys in configuration or a secret store, never in source; rotate any key that was exposed.	Thinking deleting the key from the newest commit or encoding it fixes the leak.	A key in source code or a public repository.

Course scope (public description): This course is the first in a three-course C# sequence within the BSAIE program (AI Engineering with C#, Advanced C#, and C# .NET Backend Development). Building on students’ prior experience with Python and object-oriented programming, the course transitions learners to C# as a second programming language. Students design and develop functional C# applications that apply programming logic, incorporate user interface elements, manage basic data input and output, and integrate external libraries and services, including AI-enabled components. The course emphasizes applied software engineering practices, including code modularity, control flow, and the translation of object-oriented concepts from Python to C#. AI integration is introduced at a foundational, application level, focusing on incorporating existing AI services or libraries rather than developing or training AI models. This course is not beginner-focused and assumes prior coding experience. It prepares students for continued work in backend engineering with C# while also establishing foundational AI integration skills relevant to both the BSAIE and BSSWE programs.